

MODULE 3

Process synchronization & Deadlocks

MODULE 3

- Process synchronization- Race conditions – Critical section problem – Peterson's solution, Synchronization hardware, Mutex Locks, Semaphores, Monitors – Synchronization problems - Producer Consumer, Dining Philosophers and Readers-Writers.
- Deadlocks: Necessary conditions, Resource allocation graphs, Deadlock prevention, Deadlock avoidance – Banker's algorithms, Deadlock detection, Recovery from deadlock.

Process synchronization

- A cooperating process is one that can affect or be affected by other processes executing in the system.
- Cooperating processes share its code and data with other processes.
- Concurrent access to shared data may result in data inconsistency.
- There are various mechanisms to ensure the orderly execution of cooperating processes that share a logical address space, so that data consistency is maintained.

Race condition

- Example: producer-consumer problem
- The code for producer process

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

The code for the consumer process can be modified as follows:

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
  
    /* consume the item in next_consumed */  
}
```

Race condition

- We allowed both processes to manipulate the variable counter concurrently.
- A situation like this, where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.
- To guard against the race condition , we need to ensure that only one process at a time can be manipulating the variable counter.
- To make such a guarantee, we require that the processes be synchronized in some way.

The Critical-Section Problem

- Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$.
- Each process has a segment of code, called a critical section, in which the process may be changing common variables, updating a table, writing a file, and so on.
- When one process is executing in its critical section, no other process is allowed to execute in its critical section. That is, **no two processes are executing in their critical sections at the same time.**

Critical-section problem

- The critical-section problem is to design a protocol that the processes can use to cooperate.
- Each process must request permission to enter its critical section.
- The section of code implementing this request is the entry section.
- The critical section may be followed by an exit section. The remaining code is the remainder section.

General structure of a typical process P_i

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (true);
```

Figure 5.1 General structure of a typical process P_i .

Critical-section problem

- A solution to the critical-section problem must satisfy the following three requirements:
 1. Mutual exclusion - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
 2. Progress - If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

Critical-section problem

3. Bounded waiting - There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

- It is up to kernel developers to ensure that the operating system is free from such race conditions.
- Two general approaches are used to handle critical sections in operating systems: preemptive kernels and nonpreemptive kernels.
- A preemptive kernel allows a process to be preempted while it is running in kernel mode.

Critical-section problem

- A nonpreemptive kernel does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.
- A nonpreemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time.
- Preemptive kernel must be carefully designed to ensure that shared kernel data are free from race conditions.
- A preemptive kernel may be more responsive and more suitable for real-time programming.

Peterson's Solution

- A classic software-based solution to the critical-section problem known as Peterson's solution.
- There are no guarantees that Peterson's solution will work correctly on modern computer architectures.
- However, we present the solution because it provides a good algorithmic description of solving the critical-section problem .
- Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections.

Peterson's Solution

- The processes are numbered P_0 and P_1 . For convenience, when presenting P_i , we use P_j to denote the other process.
- Peterson's solution requires the two processes to share two data items:

```
int turn;  
boolean flag[2];
```
- The variable `turn` indicates whose turn it is to enter its critical section. That is, if `turn == i`, then process P_i is allowed to execute in its critical section.
- The flag array is used to indicate if a process is ready to enter its critical section. For example, if `flag[i]` is true, this value indicates that P_i is ready to enter its critical section.

Peterson's Solution

- The structure of process P_i in Peterson's solution.

```
do {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j);  
    critical section  
    flag[i] = false;  
    remainder section  
} while (true);
```

Peterson's Solution

- To enter the critical section, process P_i first sets $flag[i]$ to be true and then sets $turn$ to the value j , thereby asserting that if the other process wishes to enter the critical section, it can do so.
- If both processes try to enter at the same time, $turn$ will be set to both i and j at roughly the same time.
- Only one of these assignments will last; the other will occur but will be overwritten immediately.
- The eventual value of $turn$ determines which of the two processes is allowed to enter its critical section first.

Peterson's Solution

- We now prove that this solution is correct. We need to show that:
 1. Mutual exclusion is preserved.
 2. The progress requirement is satisfied.
 3. The bounded-waiting requirement is met.
- To prove property 1, we note that each P_i enters its critical section only if either $\text{flag}[j] == \text{false}$ or $\text{turn} == i$.
- Also note that, if both processes can be executing in their critical sections at the same time, then $\text{flag}[0] == \text{flag}[1] == \text{true}$

Peterson's Solution

- It implies that P_0 and P_1 could not have successfully executed their while statements at about the same time, since the value of `turn` can be either 0 or 1 but cannot be both.
- Hence, one of the processes —say, P_j —must have successfully executed the while statement, whereas P_i had to execute at least one additional statement (`“turn == j”`).
- However, at that time, `flag[j] == true` and `turn == j`, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.

Peterson's Solution

- To prove properties 2 and 3, we note that a process P_i can be prevented from entering the critical section only if it is stuck in the while loop with the condition $\text{flag}[j] == \text{true}$ and $\text{turn} == j$; this loop is the only one possible.
- If P_j is not ready to enter the critical section, then $\text{flag}[j] == \text{false}$, and P_i can enter its critical section.
- If P_j has set $\text{flag}[j]$ to true and is also executing in its while statement, then either $\text{turn} == i$ or $\text{turn} == j$. If $\text{turn} == i$, then P_i will enter the critical section. If $\text{turn} == j$, then P_j will enter the critical section.

Peterson's Solution

- However, once P_j exits its critical section, it will reset $\text{flag}[j]$ to false, allowing P_i to enter its critical section.
- If P_j resets $\text{flag}[j]$ to true, it must also set turn to i . Thus, since P_i does not change the value of the variable turn while executing the while statement, P_i will enter the critical section (progress) after at most one entry by P_j (bounded waiting).

Synchronization Hardware

- There are several more solutions to the critical-section problem using techniques ranging from hardware to software-based APIs .
- All these solutions are based on the premise of locking —that is, protecting critical regions through the use of locks.
- There are some simple hardware instructions for solving the critical-section problem.
- Hardware features can make any programming task easier and improve system efficiency.

Synchronization Hardware

- Many modern computer systems provide special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words atomically—that is, as one uninterruptible unit.
- Eg: Test and set() and compare and swap() instructions.
- We can use these special instructions to solve the critical-section problem in a relatively simple manner.

Synchronization Hardware

Test and set() instruction

- The important characteristic of this instruction is that it is executed atomically.
- Thus, if two test and set() instructions are executed simultaneously they will be executed sequentially in some arbitrary order.
- If the machine supports the test and set() instruction, then we can implement mutual exclusion by declaring a Boolean variable lock, initialized to false.
- Process P_i wants to execute its critical section. So it needs to check whether any other process is executing in critical section. To check this process P_i executes test and set() instruction.

Synchronization Hardware

- The structure of process P_i is

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
  
    /* remainder section */  
} while (true);
```

Figure 5.4 Mutual-exclusion implementation with `test_and_set()`.

- The test and set() instruction can be defined as

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
  
    return rv;  
}
```

Figure 5.3 The definition of the `test_and_set()` instruction.

Synchronization Hardware

- Lock is a Boolean variable initialized to false.

address:1000
lock **false**

- While executing test and set(&lock) address of the lock variable(ie,1000) is passed and
 - *target = false
 - rv = false
 - *target = true (ie,lock=true)
 - Return rv (ie,false)
- After executing test and set() it returns a false value. That means no other process executing in critical section. So process Pi can enter into critical section. Note that now lock is true.

Synchronization Hardware

- Suppose process P_j also request for executing critical section.
- It executes test and set() instruction.
 - *target = true (ie, now lock is true because P_i executing in critical section)
 - rv = true
 - *target = true (ie, lock=true)
 - Return rv (ie,true)
- After executing test and set() it returns a true value. That means a process is executing in critical section. So process P_j can not enter into critical section.

Synchronization Hardware

compare and swap() instruction

- It operates on three operands.

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
  
    return temp;  
}
```

- The operand value is set to new value only if the expression (*value == expected) is true.
- compare and swap() always returns the original value of the variable value.

Synchronization Hardware

- The structure of process P_i is

```
do {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
} while (true);
```

- The compare and swap() is defined as

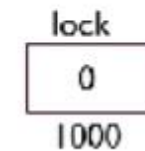
```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
  
    return temp;  
}
```

Synchronization Hardware

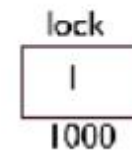
```
int compare_and_swap(int *value, int expected, int new value)
{
    int temp = *value;
    if (*value == expected)
        *value = new value;
    return temp;
}
```

```
do {
    while (compare_and_swap(&lock, 0, 1) != 0);
    /* critical section */
    lock = 0;
    /* remainder section */
} while (true);
```

lock
0



```
&lock = 1000
*1000 = FALSE
```



```
&lock = 1000
*1000 = TRUE
```

1. P1 comes for critical section.
2. Calls compare&swap function passing value of **lock**.
3. Integer **temp** is set to **0**.
4. Now it compares **lock** with **expected(0)**. Condition is true.
5. Set **lock = 1**.
6. Return the value of **temp (i.e, 0)**.
7. While condition fails. Comes out of while loop.
8. Start execution in critical section.
9. Another process P2 comes for critical section.
10. Same thing happens. But already **lock=1** So functions returns 1.
11. So P2 runs in loop until P1 completes and sets lock = 0 again.

*value(lock)=0	expected=0	new value=1	temp=" "
*value(lock)=0	expected=0	new value=1	temp=" "
*value(lock)=0	expected=0	new value=1	<u>temp=0</u>
<u>*value(lock)=0</u>	<u>expected=0</u>	new value=1	temp=0
<u>*value(lock)=1</u>	expected=0	new value=1	temp=0
*value(lock)=1	expected=0	new value=1	temp=0
*value(lock)=1	expected=0	new value=1	temp=0
*value(lock)=1	expected=0	new value=1	temp=0
*value(lock)=1	expected=0	new value=1	temp=0
*value(lock)=1	expected=0	new value=1	temp=1

Mutex Locks

- The hardware-based solutions to the critical-section problem presented are complicated.
- So operating-systems designers build software tools to solve the critical-section problem. The simplest of these tools is the mutex lock. (the term mutex is short for mutual exclusion.)
- We use the mutex lock to protect critical regions and thus prevent race conditions. That is, a process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section.

Mutex locks

The definition of `acquire()` is as follows:

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;;  
}
```

The definition of `release()` is as follows:

```
release() {  
    available = true;  
}
```

```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (true);
```

Figure 5.8 Solution to the critical-section problem using mutex locks.

Mutex locks

- A mutex lock has a boolean variable available whose value indicates if the lock is available or not.(ie,available=true, lock is available and available =false,lock is not available)
- If the lock is available, a call to acquire() succeeds, and the lock is then considered unavailable.
- A process that attempts to acquire an unavailable lock is blocked until the lock is released.
- Calls to either acquire() or release() must be performed atomically.

Mutex locks

- The main disadvantage of this implementation is that it requires busy waiting.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to `acquire()`.
- This type of mutex lock is also called a spinlock because the process “spins” while waiting for the lock to become available.
- Busy waiting wastes CPU cycles that some other process might be able to use productively.

Semaphores

- Provide more sophisticated ways for processes to synchronize their activities.
- A semaphore S is an integer variable that is accessed only through two standard atomic operations:
 `wait()` and `signal()`.
- The `wait()` operation was originally termed P and `signal()` was originally called V .

Semaphores

definition of `wait()` is as follows:

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

The definition of `signal()` is as follows:

```
signal(S) {  
    S++;  
}
```

Semaphores

- All modifications to the integer value of the semaphore in the wait() and signal() operations must be executed indivisibly.
- That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.
- In addition, in the case of wait(S), the testing of the integer value of S ($S \leq 0$), as well as its possible modification ($S--$), must be executed without interruption.

Semaphores

Semaphore Usage

- Operating systems often distinguish between counting and binary semaphores.
- The value of a counting semaphore can range over an unrestricted domain. The value of a binary semaphore can range only between 0 and 1.
- Thus, binary semaphores behave similarly to mutex locks.
- On systems that do not provide mutex locks, binary semaphores can be used instead for providing mutual exclusion.

Semaphores

- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.
- The semaphore is initialized to the number of resources available.
- Each process that wishes to use a resource performs a `wait()` operation on the semaphore (thereby decrementing the count).
- When a process releases a resource, it performs a `signal()` operation (incrementing the count).
- When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

Semaphores

- We can also use semaphores to solve various synchronization problems. For example, consider two concurrently running processes: P1 with a statement S1 and P2 with a statement S2. Suppose we require that S2 be executed only after S1 has completed.
- We can implement this scheme readily by letting P1 and P2 share a common semaphore synch, initialized to 0. In process P1, we insert the statements,

Semaphores

```
 $S_1$  ;  
signal(synch) ;
```

In process P_2 , we insert the statements

```
wait(synch) ;  
 $S_2$  ;
```

- Because `synch` is initialized to 0, P_2 will execute S_2 only after P_1 has invoked `signal(synch)`, which is after statement S_1 has been executed.

Semaphores

Semaphore Implementation

- To overcome the need for busy waiting, we can modify the definition of the wait() and signal() operations as follows: When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, the process can block itself. The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Then control is transferred to the CPU scheduler, which selects another process to execute.

Semaphores

- A process that is blocked, waiting on a semaphore S , should be restarted when some other process executes a `signal()` operation.
- The process is restarted by a `wakeup()` operation, which changes the process from the waiting state to the ready state.
- The process is then placed in the ready queue.
- To implement semaphores under this definition, we define a semaphore as follows:

Semaphores

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

- Each semaphore has an integer value and a list of processes list. When a process must wait on a semaphore, it is added to the list of processes.
- A signal() operation removes one process from the list of waiting processes and awakens that process.

Semaphores

- The wait() semaphore operation can be defined as

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

- The signal() semaphore operation can be defined as

and the signal() semaphore operation can be defined as

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Semaphores

- The `block()` operation suspends the process that invokes it. The `wakeup(P)` operation resumes the execution of a blocked process `P`.
- These two operations are provided by the operating system as basic system calls.
- In this implementation, semaphore values may be negative, whereas semaphore values are never negative under the classical definition of semaphores with busy waiting.
- If a semaphore value is negative, its magnitude is the number of processes waiting on that semaphore.

Deadlocks and Starvation

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes. When such a state is reached, these processes are said to be deadlocked.
- consider a system consisting of two processes, P0 and P1, each accessing two semaphores, S and Q, set to the value 1:

Deadlocks and Starvation

P_0	P_1
<code>wait(S);</code>	<code>wait(Q);</code>
<code>wait(Q);</code>	<code>wait(S);</code>
<code>.</code>	<code>.</code>
<code>.</code>	<code>.</code>
<code>.</code>	<code>.</code>
<code>signal(S);</code>	<code>signal(Q);</code>
<code>signal(Q);</code>	<code>signal(S);</code>

- Suppose that P_0 executes `wait(S)` and then P_1 executes `wait(Q)`. When P_0 executes `wait(Q)`, it must wait until P_1 executes `signal(Q)`. Similarly, when P_1 executes `wait(S)`, it must wait until P_0 executes `signal(S)`. Since these `signal()` operations cannot be executed, P_0 and P_1 are deadlocked.

Deadlocks and Starvation

- We say that a set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.
- Another problem related to deadlocks is indefinite blocking or starvation, a situation in which processes wait indefinitely within the semaphore.

Synchronization problems

- **1 .The Bounded-Buffer Problem**

- Producer-consumer problem with bounded buffer
- The producer and consumer processes share the following data structures:
 - int n;
 - semaphore mutex = 1;
 - semaphore empty = n;
 - semaphore full = 0;
- We assume that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1.

Synchronization problems

- The empty and full semaphores count the number of empty and full buffers. The semaphore empty is initialized to the value n ; the semaphore full is initialized to the value 0.
- The code for the producer process is

```
do {  
    . . .  
    /* produce an item in next_produced */  
    . . .  
    wait(empty);  
    wait(mutex);  
    . . .  
    /* add next_produced to the buffer */  
    . . .  
    signal(mutex);  
    signal(full);  
} while (true);
```

Synchronization problems

- The code for the consumer process is

```
do {
    wait(full);
    wait(mutex);
    . . .
    /* remove an item from buffer to next_consumed */
    . . .
    signal(mutex);
    signal(empty);
    . . .
    /* consume the item in next_consumed */
    . . .
} while (true);
```

Synchronization problems

Eg: n is the buffer size.

Assume $n=5$

Initial value of semaphore mutex = 1,
empty = 5 and full=0

Assume that consumer process executes first.

It execute wait(full)

The value of semaphore full=0. ie,It enters into busy wait and the consumer process cannot continue its execution.

definition of wait() is as follows:

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

The definition of signal() is as follows:

```
signal(S) {  
    S++;  
}
```

Synchronization problems

- Next producer process executes

Wait(empty) – empty=4

Wait(mutex) – mutex=0

Item is added to buffer

Signal(mutex) – mutex=1

Signal(full) – full=1

- Consumer executes

Wait(full) – full=0

Wait(mutex) – mutex=0

Item is removed from buffer

Signal(mutex) – mutex=1

Signal(empty) – empty=5

Synchronization problems

2. The Readers–Writers Problem

- Suppose that a database is to be shared among several concurrent processes. Some of these processes may want only to read the database(reader), whereas others may want to update the database(writer).
- If two readers access the shared data simultaneously, no adverse effects will result.
- If a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may arise.

Synchronization problems

- We require that the writers have exclusive access to the shared database while writing to the database. This synchronization problem is referred to as the readers–writers problem.
- The readers–writers problem has several variations, all involving priorities.
- first readers–writers problem - requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object. In other words, no reader should wait for other readers to finish simply because a writer is waiting.
- The second readers–writers problem - requires that, once a writer is ready, that writer perform its write as soon as possible. In other words, if a writer is waiting to access the object, no new readers may start reading.

Synchronization problems

- A solution to either problem may result in starvation. In the first case, writers may starve; in the second case, readers may starve.
- In the solution to the first readers–writers problem, the reader processes share the following data structures:

```
semaphore rw_mutex = 1;  
semaphore mutex = 1;  
int read count = 0;
```
- The semaphores mutex and rw_mutex are initialized to 1; read count is initialized to 0. The semaphore rw_mutex is common to both reader and writer processes.

Synchronization problems

- The mutex semaphore is used to ensure mutual exclusion when the variable read count is updated.
- The read count variable keeps track of how many processes are currently reading the object.
- The semaphore `rw_mutex` functions as a mutual exclusion semaphore for the writers.
- It is also used by the first or last reader that enters or exits the critical section. It is not used by readers who enter or exit while other readers are in their critical sections.

Synchronization problems

- The code for a writer process is

```
do {  
    wait(rw_mutex);  
    . . .  
    /* writing is performed */  
    . . .  
    signal(rw_mutex);  
} while (true);
```

Synchronization problems

- The code for a reader process is

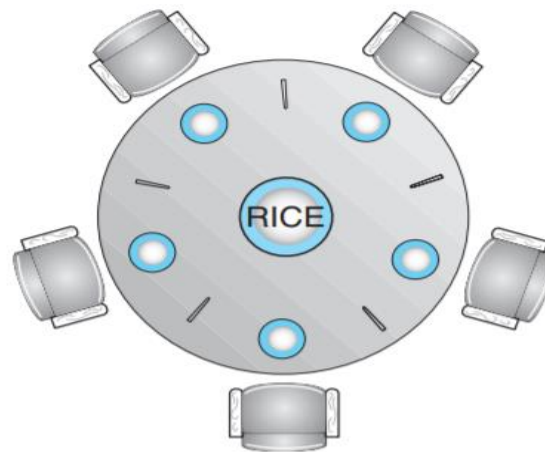
```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    . . .
    /* reading is performed */
    . . .
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Synchronization problems

3) The Dining-Philosophers Problem

- Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chopsticks.



Synchronization problems

- When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors).
- A philosopher may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick that is already in the hand of a neighbor.
- When a hungry philosopher has both her chopsticks at the same time, she eats without releasing the chopsticks. When she is finished eating, she puts down both chopsticks and starts thinking again.

Synchronization problems

- The dining philosopher problem is a simple representation of the need to allocate several resources among several processes in a deadlock-free and starvation-free manner.
- One simple solution is to represent each chopstick with a semaphore.
- A philosopher tries to grab a chopstick by executing a `wait()` operation on that semaphore.
- She releases her chopsticks by executing the `signal()` operation on the appropriate semaphores.

Synchronization problems

- The shared data are ,
 semaphore chopstick[5];
 where all the elements of chopstick are initialized to 1.
- The structure of philosopher i is shown

```
do {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
    . . .  
    /* eat for awhile */  
    . . .  
    signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
    . . .  
    /* think for awhile */  
    . . .  
} while (true);
```

Synchronization problems

- This solution guarantees that no two neighbors are eating simultaneously, it nevertheless must be rejected because it could create a deadlock.
- Suppose that all five philosophers become hungry at the same time and each grabs her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.
- Several possible remedies to the deadlock problem are replaced by:

Synchronization problems

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even numbered philosopher picks up her right chopstick and then her left chopstick.

Monitors

- Although semaphores provide a convenient and effective mechanism for process synchronization, using them incorrectly can result in timing errors that are difficult to detect, since these errors happen only if particular execution sequences take place and these sequences do not always occur.
- Eg: consider the semaphore solution to the critical-section problem.
- All processes share a semaphore variable mutex, which is initialized to 1.
- Each process must execute wait(mutex) before entering the critical section and signal(mutex) afterward.
- If this sequence is not observed, two processes may be in their critical sections simultaneously.

Monitors

- Various difficulties will arise even if a single process is not well behaved. This situation may be caused by an honest programming error or an uncooperative programmer.
- Suppose that a process interchanges the order in which the wait() and signal() operations on the semaphore mutex are executed, resulting in the following execution:

```
signal(mutex);  
...  
critical section  
...  
wait(mutex);
```

Monitors

- In this situation, several processes may be executing in their critical sections simultaneously, violating the mutual-exclusion requirement. This error may be discovered only if several processes are simultaneously active in their critical sections.
- Suppose that a process replaces `signal(mutex)` with `wait(mutex)`. That is, it executes

```
wait(mutex);  
...  
critical section  
...  
wait(mutex);
```

In this case, a deadlock will occur.

Monitors

- Suppose that a process omits the wait(mutex), or the signal(mutex), or both. In this case, either mutual exclusion is violated or a deadlock will occur.
- To deal with such errors, researchers have developed high-level language constructs such as—the monitor type.

Monitor Usage

- Monitor is an abstract data type—or ADT—encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of the ADT.
- A monitor type is an ADT that includes a set of programmer defined operations that are provided with mutual exclusion within the monitor.

Monitors

- The monitor type also declares the variables whose values define the state of an instance of that type, along with the bodies of functions that operate on those variables.
- The syntax of a monitor type is

```
monitor monitor name
{
    /* shared variable declarations */

    function P1 ( . . . ) {
        . . .
    }

    function P2 ( . . . ) {
        . . .
    }

    .
    .
    .
    function Pn ( . . . ) {
        . . .
    }

    initialization_code ( . . . ) {
        . . .
    }
}
```

Monitors

- The representation of a monitor type cannot be used directly by the various processes.
- Thus, a function defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.
- Similarly, the local variables of a monitor can be accessed by only the local functions.
- The monitor construct ensures that only one process at a time is active within the monitor.
- Consequently, the programmer does not need to code this synchronization constraint explicitly .

Monitors

- Schematic view of a monitor

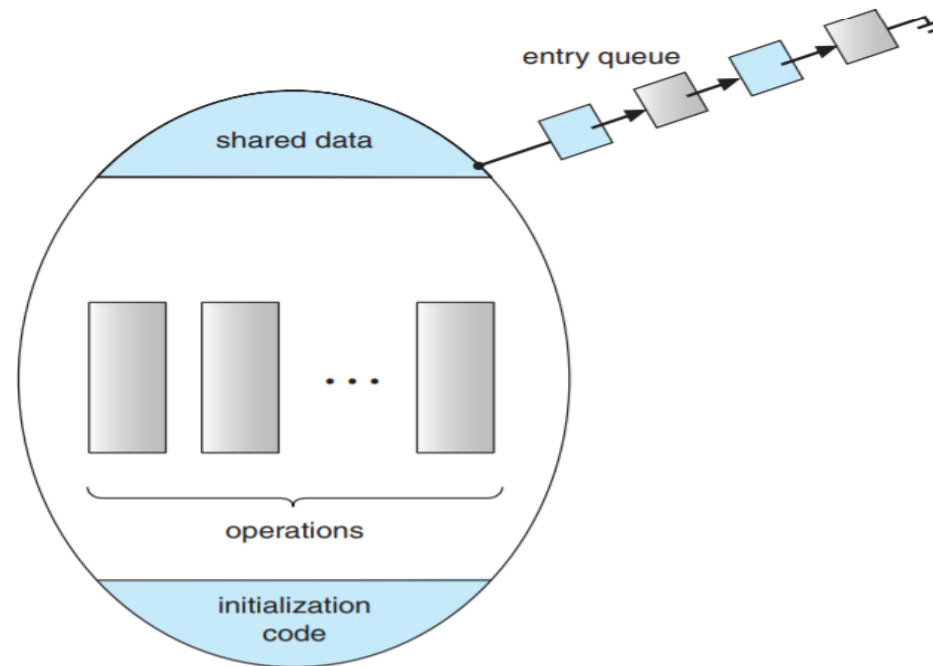


Figure 5.16 Schematic view of a monitor.

Monitors

- A programmer who needs to write a tailor-made synchronization scheme can define one or more variables of type condition:

condition x, y;

- The only operations that can be invoked on a condition variable are wait() and signal(). The operation

x.wait();

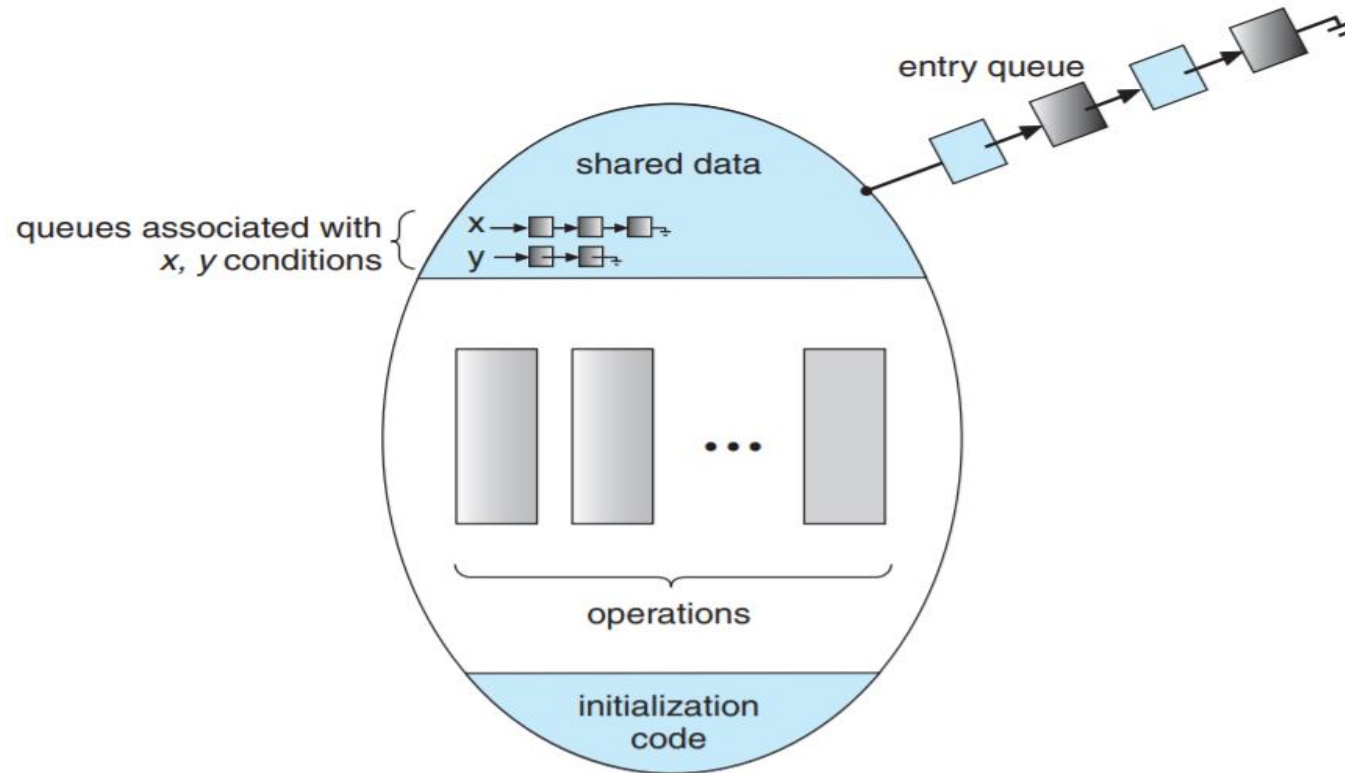
means that the process invoking this operation is suspended until another process invokes

x.signal();

- The x.signal() operation resumes exactly one suspended process. If no process is suspended, then the signal() operation has no effect; that is, the state of x is the same as if the operation had never been executed. Contrast this operation with the signal() operation associated with semaphores, which always affects the state of the semaphore.

Monitors

- Monitor with condition variables



Monitors

- Now suppose that, when the `x.signal()` operation is invoked by a process `P`, there exists a suspended process `Q` associated with condition `x`.
- If the suspended process `Q` is allowed to resume its execution, the signaling process `P` must wait.
- Otherwise, both `P` and `Q` would be active simultaneously within the monitor. Note, however, that conceptually both processes can continue with their execution. Two possibilities exist:
 - 1.Signal and wait. `P` either waits until `Q` leaves the monitor or waits for another condition.
 2. Signal and continue. `Q` either waits until `P` leaves the monitor or waits for another condition.
- Since `P` was already executing in the monitor, the signal-and continue method seems more reasonable.

Deadlocks

- In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources; if the resources are not available at that time, the process enters a waiting state.
- Sometimes, a waiting process is never again able to change state, because the resources it has requested are held by other waiting processes. This situation is called a deadlock.
- A process must request a resource before using it and must release the resource after using it.
- A process may request as many resources as it requires to carry out its designated task.
- The number of resources requested may not exceed the total number of resources available in the system.

Deadlocks

- Under the normal mode of operation, a process may utilize a resource in the following sequence:
 1. Request. The process requests the resource. If the request cannot be granted immediately (for example, if the resource is being used by another process), then the requesting process must wait until it can acquire the resource.
 2. Use. The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).
 3. Release. The process releases the resource.

Deadlocks

- The request and release of resources may be system calls.
- The operating system checks to make sure that the process has requested and has been allocated the resource.
- A system table records whether each resource is free or allocated. For each resource that is allocated, the table also records the process to which it is allocated.
- If a process requests a resource that is currently allocated to another process, it can be added to a queue of processes waiting for this resource.
- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set. The events with which we are mainly concerned here are resource acquisition and release.

Necessary Conditions

- A deadlock situation can arise if the following four conditions hold simultaneously in a system:
 1. **Mutual exclusion**- At least one resource must be held in a non-sharable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
 2. **Hold and wait**-A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

Necessary Conditions

3. No preemption- Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.

4. Circular wait- A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for a resource held by P_2 , ..., P_{n-1} is waiting for a resource held by P_n , and P_n is waiting for a resource held by P_0 .

Resource-Allocation Graph

- Deadlocks can be described more precisely in terms of a directed graph called a system resource-allocation graph.
- This graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes: $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system, and $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.
- A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$; it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource. A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$; it signifies that an instance of resource type R_j has been allocated to process P_i . A directed edge $P_i \rightarrow R_j$ is called a request edge; a directed edge $R_j \rightarrow P_i$ is called an assignment edge.

Resource-Allocation Graph

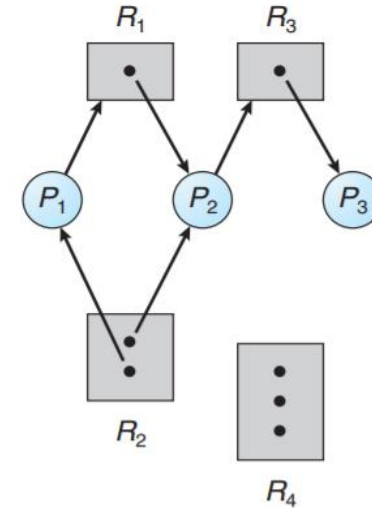
- Pictorially, we represent each process P_i as a circle and each resource type R_j as a rectangle.
- Since resource type R_j may have more than one instance, we represent each such instance as a dot within the rectangle.
- Note that a request edge points to only the rectangle R_j , whereas an assignment edge must also designate one of the dots in the rectangle.
- When process P_i requests an instance of resource type R_j , a request edge is inserted in the resource-allocation graph.
- When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge.
- When the process no longer needs access to the resource, it releases the resource. As a result, the assignment edge is deleted.

Resource-Allocation Graph

- It represents
 - The sets P, R, and E:
 - $P = \{P_1, P_2, P_3\}$
 - $R = \{R_1, R_2, R_3, R_4\}$
 - $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_1, R_2 \rightarrow P_2, R_3 \rightarrow P_3\}$

Resource instances:

- One instance of resource type R1
- Two instances of resource type R2 ◦
- One instance of resource type R3 ◦
- Three instances of resource type R4

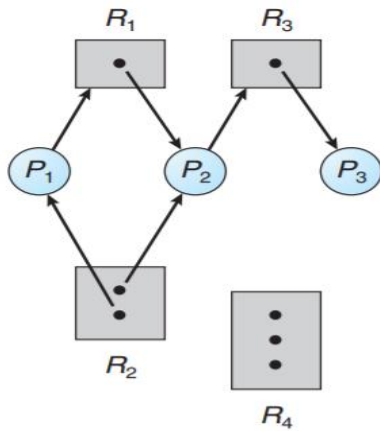


Resource-Allocation Graph

- Process states:
 - Process P1 is holding an instance of resource type R2 and is waiting for an instance of resource type R1.
 - Process P2 is holding an instance of R1 and an instance of R2 and is waiting for an instance of R3.
 - Process P3 is holding an instance of R3.
- If the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.
- If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. In this case a cycle in the graph is both a necessary and a sufficient condition for of deadlock.

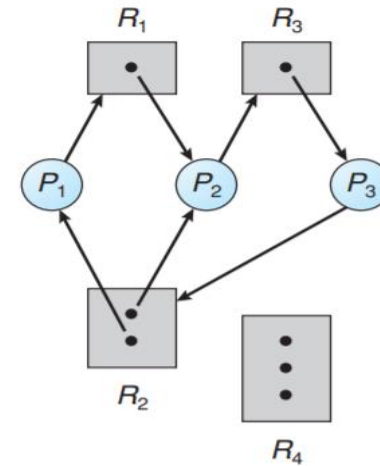
Resource-Allocation Graph

- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.
- Eg: Consider the graph



Resource-Allocation Graph

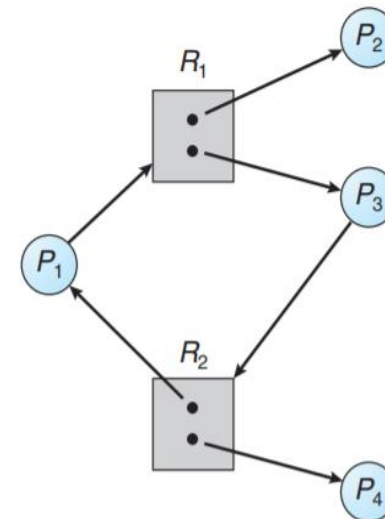
- Suppose that process P_3 requests an instance of resource type R_2 . Since no resource instance is currently available, we add a request edge $P_3 \rightarrow R_2$ to the graph.



- At this point, two minimal cycles exist in the system:
- $P_1 \rightarrow R_1 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$
- $P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_2 \rightarrow P_2$

Resource-Allocation Graph

- Processes P1, P2, and P3 are deadlocked.
- Process P2 is waiting for the resource R3, which is held by process P3.
- Process P3 is waiting for either process P1 or process P2 to release resource R2.
- Process P1 is waiting for process P2 to release resource R1.
- Consider the resource-allocation graph



- we have a cycle: $P1 \rightarrow R1 \rightarrow P3 \rightarrow R2 \rightarrow P1$

Resource-Allocation Graph

- We have a cycle: $P1 \rightarrow R1 \rightarrow P3 \rightarrow R2 \rightarrow P1$
- There is no deadlock.
- Observe that process P4 may release its instance of resource type R2.
- That resource can then be allocated to P3, breaking the cycle.
- In summary, if a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state. If there is a cycle, then the system may or may not be in a deadlocked state.

Methods for Handling Deadlocks

- We can deal with the deadlock problem in one of three ways:
 - We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
 - We can allow the system to enter a deadlocked state, detect it, and recover.
 - We can ignore the problem altogether and pretend that deadlocks never occur in the system.
- Deadlock prevention provides a set of methods to ensure that at least one of the necessary conditions cannot hold.

Methods for Handling Deadlocks

- Deadlock avoidance requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
- With this additional knowledge, the operating system can decide for each request whether or not the process should wait.
- To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.

Deadlock Prevention

- For a deadlock to occur, each of the four necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.

Mutual Exclusion

- The mutual exclusion condition must hold. That is, at least one resource must be nonsharable.
- Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock. Read-only files are a good example of a sharable resource.
- If several processes attempt to open a read-only file at the same time, they can be granted simultaneous access to the file. A process never needs to wait for a sharable resource.

Deadlock Prevention

- We cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically nonsharable. For example, a mutex lock cannot be simultaneously shared by several processes.

Hold and Wait

- To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a process requests a resource, it does not hold any other resources.
- One protocol that we can use requires each process to request and be allocated all its resources before it begins execution.

Deadlock Prevention

- An alternative protocol allows a process to request resources only when it has none. A process may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.
- Both these protocols have two main disadvantages. First, resource utilization may be low, since resources may be allocated but unused for a long period.
- Second, starvation is possible. A process that needs several popular resources may have to wait indefinitely, because at least one of the resources that it needs is always allocated to some other process.

Deadlock Prevention

No Preemption

- To ensure that this condition does not hold, we can use the following protocol. If a process is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the process must wait), then all resources the process is currently holding are preempted.
- ie, these resources are implicitly released. The preempted resources are added to the list of resources for which the process is waiting.
- The process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Deadlock Prevention

- Alternatively, if a process requests some resources, we first check whether they are available. If they are, we allocate them. If they are not, we check whether they are allocated to some other process that is waiting for additional resources.
- If so, we preempt the desired resources from the waiting process and allocate them to the requesting process. If the resources are neither available nor held by a waiting process, the requesting process must wait.
- While it is waiting, some of its resources may be preempted, but only if another process requests them.
- A process can be restarted only when it is allocated the new resources it is requesting and recovers any resources that were preempted while it was waiting.

Deadlock Prevention

Circular Wait

- One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.
- A process can initially request any number of instances of a resource type —say, R_i . After that, the process can request instances of resource type R_j if and only if $F(R_j) > F(R_i)$.
- Alternatively, we can require that a process requesting an instance of resource type R_j must have released any resources R_i such that $F(R_i) \geq F(R_j)$. If these two protocols are used, then the circular-wait condition cannot hold.

Deadlock Avoidance

- The side effects of deadlock prevention are low device utilization and reduced system throughput. An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested.
- Eg: In a system with one tape drive and one printer, the system might need to know that ,
 - process P will request first the tape drive and then the printer
 - process Q will request first the printer and then the tape drive.
- With this knowledge of the complete sequence of requests and releases for each process, the system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock.

Deadlock Avoidance

- Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.
- Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state.

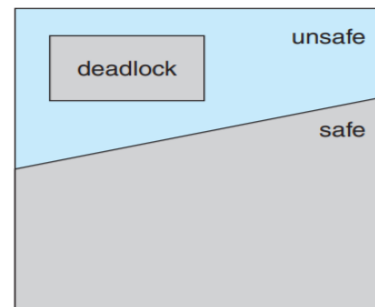
Deadlock-avoidance algorithms

Safe state

- A system is in a safe state only if there exists a safe sequence.
- A sequence of processes is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$.
- If the resources that P_i needs are not immediately available, then P_i can wait until all P_j have finished.
- When they have finished, P_i can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate.

Deadlock-avoidance algorithms

- When P_i terminates, P_{i+1} can obtain its needed resources, and so on.
- If no such sequence exists, then the system state is said to be unsafe.
- A safe state is not a deadlocked state. Conversely, a deadlocked state is an unsafe state. Not all unsafe states are deadlocks. An unsafe state may lead to a deadlock.



i Safe, unsafe, and deadlocked state spaces.

Deadlock-avoidance algorithms

- Eg: Consider a system with twelve magnetic tape drives and three processes: P0, P1, and P2.
- Process P0 requires ten tape drives, process P1 may need as many as four tape drives, and process P2 may need up to nine tape drives.
- At time t0, process P0 is holding five tape drives, process P1 is holding two tape drives, and process P2 is holding two tape drives. (Thus, there are three free tape drives.)

	Maximum Needs	Current Needs
• P0	10	5
• P1	4	2
• P2	9	2

Deadlock-avoidance algorithms

- At time t_0 , the system is in a safe state.
- The sequence P1, P0, P2 satisfies the safety condition.
- Process P1 can immediately be allocated all its tape drives and then return them (the system will then have five available tape drives); then process P0 can get all its tape drives and return them (the system will then have ten available tape drives); and finally process P2 can get all its tape drives and return them (the system will then have all twelve tape drives available).

Deadlock-avoidance algorithms

- A system can go from a safe state to an unsafe state.
- Suppose that, at time t1, process P2 requests and is allocated one more tape drive.

	Maximum Needs	Current Needs
P0	10	5
P1	4	2
P2	9	3

- The system is no longer in a safe state. At this point, only process P1 can be allocated all its tape drives.
When it returns them, the system will have only four available tape drives.

Deadlock-avoidance algorithms

- Since process P0 is allocated five tape drives but has a maximum of ten, it may request five more tape drives. If it does so, it will have to wait, because they are unavailable.
- Process P2 may request six additional tape drives and have to wait, resulting in a deadlock.
- If we had made *P2 wait* until either of the other processes had finished and released its resources, then we could have avoided the deadlock.

Deadlock-avoidance algorithms

- Given the concept of a safe state, we can define avoidance algorithms that ensure that the system will never deadlock.
- The idea is simply to ensure that the system will always remain in a safe state.
- Initially, the system is in a safe state. Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or whether the process must wait.
- The request is granted only if the allocation leaves the system in a safe state.
- In this scheme, if a process requests a resource that is currently available, it may still have to wait.

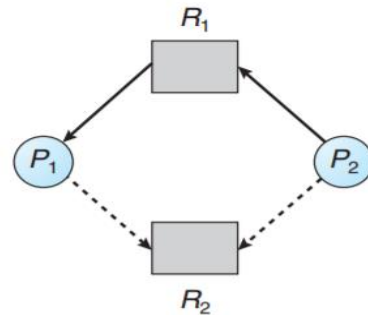
Deadlock-avoidance algorithms

Resource-Allocation-Graph Algorithm

- If we have a resource-allocation system with only one instance of each resource type, we can use a variant of the resource-allocation graph for deadlock avoidance.
- In addition to the request and assignment edges already described, we introduce a new type of edge, called a **claim edge**.
- A claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future.
- This edge resembles a request edge in direction but is represented in the graph by a dashed line.
- When process P_i requests resource R_j , the claim edge $P_i \rightarrow R_j$ is converted to a request edge.
- When a resource R_j is released by P_i , the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$.

Deadlock-avoidance algorithms

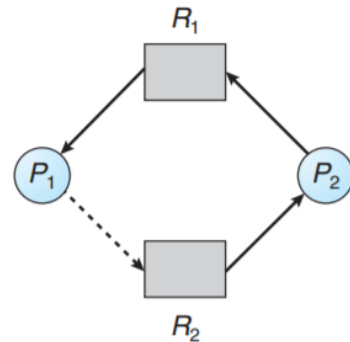
- Suppose that process P_i requests resource R_j .
- The request can be granted only if converting the request edge $P_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow P_i$ does not result in the formation of a cycle in the resource-allocation graph.
- If no cycle exists, then the allocation of the resource will leave the system in a safe state.
- If a cycle is found, then the allocation will put the system in an unsafe state.



Resource-allocation graph for deadlock avoidance.

Deadlock-avoidance algorithms

- Suppose that P2 requests R2.
- Although R2 is currently free, we cannot allocate it to P2, since this action will create a cycle in the graph .



An unsafe state in a resource-allocation graph.

- A cycle, as mentioned, indicates that the system is in an unsafe state. If P_1 requests R_2 , and P_2 requests R_1 , then a deadlock will occur.

Deadlock-avoidance algorithms

Banker's Algorithm

- The resource-allocation-graph algorithm is not applicable to a resource allocation system with multiple instances of each resource type.
- Banker's algorithm is used in such situations.
- When a new process enters the system, it must declare the maximum number of instances of each resource type that it may need.
- This number may not exceed the total number of resources in the system.
- When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the process must wait until some other process releases enough resources.

Deadlock-avoidance algorithms

- Several data structures must be maintained to implement the banker's algorithm.
 - n - the number of processes in the system
 - m - the number of resource types
- Available - A vector of length m indicates the number of available resources of each type. If Available[j] equals k , then k instances of resource type R_j are available.
- Max - An $n \times m$ matrix defines the maximum demand of each process. If Max[i][j] equals k , then process P_i may request at most k instances of resource type R_j .

Deadlock-avoidance algorithms

- Allocation - An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i][j]$ equals k , then process P_i is currently allocated k instances of resource type R_j .
- Need - An $n \times m$ matrix indicates the remaining resource need of each process. If $\text{Need}[i][j]$ equals k , then process P_i may need k more instances of resource type R_j to complete its task. Note that $\text{Need}[i][j]$ equals $\text{Max}[i][j] - \text{Allocation}[i][j]$.

Deadlock-avoidance algorithms

- Let X and Y be vectors of length n .
- We say that $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all $i = 1, 2, \dots, n$.
- For example, if $X = (1, 7, 3, 2)$ and $Y = (0, 3, 2, 1)$, then $Y \leq X$.
- In addition, $Y < X$ if $Y \leq X$ and $Y \neq X$.
- We can treat each row in the matrices Allocation and Need as vectors and refer to them as Allocation_i and Need_i .
- The vector Allocation_i specifies the resources currently allocated to process P_i ; the vector Need_i specifies the additional resources that process P_i may still request to complete its task.

Deadlock-avoidance algorithms

- **Safety Algorithm** -the algorithm for finding out whether or not a system is in a safe state.
 1. Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available and Finish[i] = false for $i = 0, 1, \dots, n - 1$.
 2. Find an index i such that both
 - a. Finish[i] == false
 - b. $\text{Need}_i \leq \text{Work}$If no such i exists, go to step 4.
 3. $\text{Work} = \text{Work} + \text{Allocation}_i$
Finish[i] = true
Go to step 2.
- 4. If Finish[i] == true for all i, then the system is in a safe state.

Deadlock-avoidance algorithms

- **Resource-Request Algorithm-** The algorithm for determining whether requests can be safely granted.
- Let $Request_i$ be the request vector for process P_i . If $Request_i[j] == k$, then process P_i wants k instances of resource type R_j . When a request for resources is made by process P_i , the following actions are taken:
 1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise an error condition, since the process has exceeded its maximum claim.
 2. If $Request_i \leq Available$, go to step 3. Otherwise, P_i must wait, since the resources are not available.

Deadlock-avoidance algorithms

3. Have the system pretend to have allocated the requested resources to

process P_i by modifying the state as follows:

$Available = Available - Request_i;$

$Allocation_i = Allocation_i + Request_i;$

$Need_i = Need_i - Request_i;$

- If the resulting resource-allocation state is safe, the transaction is completed, and process P_i is allocated its resources. However, if the new state is unsafe, then P_i must wait for $Request_i$, and the old resource-allocation state is restored.

Deadlock-avoidance algorithms

Example

- Consider a system with five processes *P0 through P4* and three resource types *A, B, and C*.
- *Resource type A* has ten instances, resource type *B* has five instances, and resource type *C* has seven instances.
- Suppose that, at time *T0*, the following snapshot of the system has been taken:

Deadlock-avoidance algorithms

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Deadlock-avoidance algorithms

- The content of the matrix *Need* is defined to be $Max - Allocation$ and is as follows:

	<i>Need</i>		
	<i>A</i>	<i>B</i>	<i>C</i>
<i>P0</i>	7	4	3
<i>P1</i>	1	2	2
<i>P2</i>	6	0	0
<i>P3</i>	0	1	1
<i>P4</i>	4	3	1

Deadlock-avoidance algorithms

- We claim that the system is currently in a safe state. Indeed, the sequence $\langle P1, P3, P4, P2, P0 \rangle$ satisfies the safety criteria.
- Suppose now that process P1 requests one additional instance of resource type A and two instances of resource type C, so $\text{Request}_1 = (1,0,2)$.
- To decide whether this request can be immediately granted, we first check that $\text{Request}_1 \leq \text{Available}$ —that is, that $(1,0,2) \leq (3,3,2)$, which is true.
- We then pretend that this request has been fulfilled, and we arrive at the following new state:

Deadlock-avoidance algorithms

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Deadlock-avoidance algorithms

- To determine whether this new system state is safe ,we execute our safety algorithm and find that the sequence $\langle P1, P3, P4, P0, P2 \rangle$ satisfies the safety requirement. Hence, we can immediately grant the request of process P1.
- When the system is in this state, a request for (3,3,0) by P4 cannot be granted, since the resources are not available.

Deadlock Detection

Deadlock Detection

- If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system may provide:
 - An algorithm that examines the state of the system to determine whether a deadlock has occurred
 - An algorithm to recover from the deadlock

Single Instance of Each Resource Type

- If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource-allocation graph, called a wait-for graph. We obtain this graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.

Deadlock Detection

- An edge from P_i to P_j in a wait-for graph implies that process P_i is waiting for process P_j to release a resource that P_i needs.
- An edge $P_i \rightarrow P_j$ exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for some resource R_q .
- A deadlock exists in the system if and only if the wait-for graph contains a cycle. To detect deadlocks, the system needs to maintain the wait for graph and periodically invoke an algorithm that searches for a cycle in the graph.

Deadlock Detection

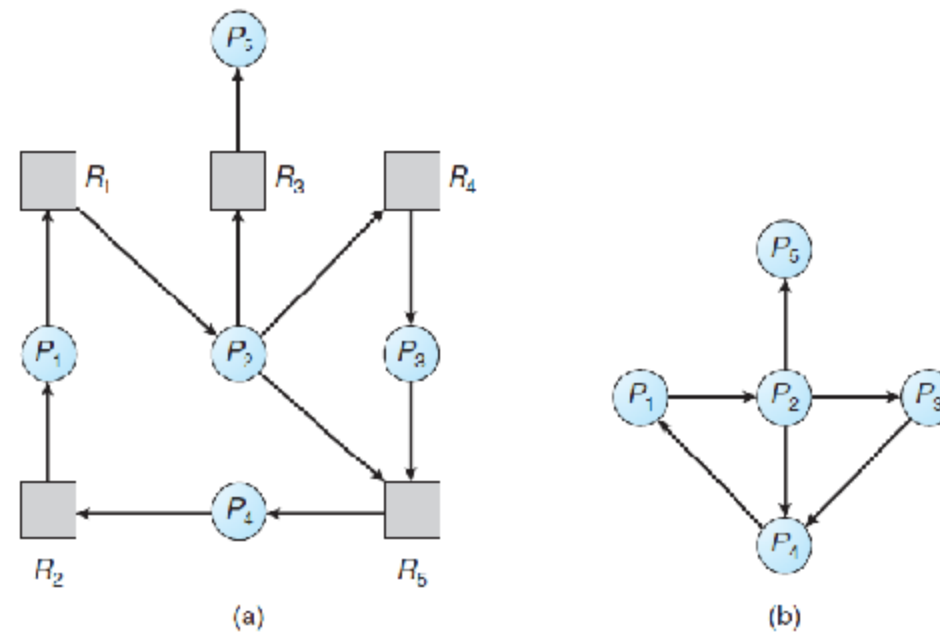


Figure 7.9 (a) Resource-allocation graph. (b) Corresponding wait-for graph.

Deadlock Detection

Several Instances of a Resource Type

- The data structures used are,
 - Available – A vector of length m indicates the number of available resources of each type.
 - Allocation - An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
 - Request - An $n \times m$ matrix indicates the current request of each process. If $\text{Request}[i][j]$ equals k , then process P_i is requesting k more instances of resource type R_j .

Deadlock Detection

1. Let Work and Finish be vectors of length m and n, respectively. Initialize Work = Available . For $i = 0, 1, \dots, n-1$, if $\text{Allocation}_i = 0$, then $\text{Finish}[i] = \text{false}$. Otherwise, $\text{Finish}[i] = \text{true}$.
2. Find an index i such that both
 - a. $\text{Finish}[i] == \text{false}$
 - b. $\text{Request}_i \leq \text{Work}$If no such i exists, go to step 4.
3. $\text{Work} = \text{Work} + \text{Allocation}_i$
 $\text{Finish}[i] = \text{true}$
Go to step 2.
4. If $\text{Finish}[i] == \text{false}$ for some i, $0 \leq i < n$, then the system is in a deadlocked state. Moreover, if $\text{Finish}[i] == \text{false}$, then process P_i is deadlocked.

Deadlock Detection

Example

- Consider a system with five processes P0 through P4 and three resource types A, B, and C. Resource type A has seven instances, resource type B has two instances, and resource type C has six instances.
- Suppose that, at time T0, we have the following resource-allocation state:

	Allocation	Request	Available
	A B C	A B C	A B C
• P0	0 1 0	0 0 0	0 0 0
• P1	2 0 0	2 0 2	
• P2	3 0 3	0 0 0	
• P3	2 1 1	1 0 0	
• P4	0 0 2	0 0 2	

Deadlock Detection

- We claim that the system is not in a deadlocked state. If we execute our algorithm, we will find that the sequence $\langle P0, P2, P3, P1, P4 \rangle$ results in $Finish[i] == true$ for all i .
- Suppose now that process $P2$ makes one additional request for an instance of type C . The Request matrix is modified as follows:

Request

	<i>A</i>	<i>B</i>	<i>C</i>
P0	0	0	0
P1	2	0	2
P2	0	0	1
P3	1	0	0
P4	0	0	2

- We claim that the system is now deadlocked.

Recovery from Deadlock

- **Process Termination**
- To eliminate deadlocks by aborting a process, we use one of two methods. In both methods, the system reclaims all resources allocated to the terminated processes.
- **Abort all deadlocked processes** - The deadlocked processes may have computed for a long time, and the results of these partial computations must be discarded and probably will have to be recomputed later.
- **Abort one process at a time until the deadlock cycle is eliminated** - This method incurs considerable overhead, since after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked.

Recovery from Deadlock

- **Resource Preemption**

- To eliminate deadlocks using resource preemption, we successively preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

- three issues need to be addressed:

1. Selecting a victim.

Which resources and which processes are to be preempted? As in process termination, we must determine the order of preemption to minimize cost

2. Rollback.

If we preempt a resource from a process, what should be done with that process? Clearly, it cannot continue with its normal execution; it is missing some needed resource. We must roll back the process to some safe state and restart it from that state.

Recovery from Deadlock

- Starvation.

How do we ensure that starvation will not occur? That is, how can we guarantee that resources will not always be preempted from the same process?